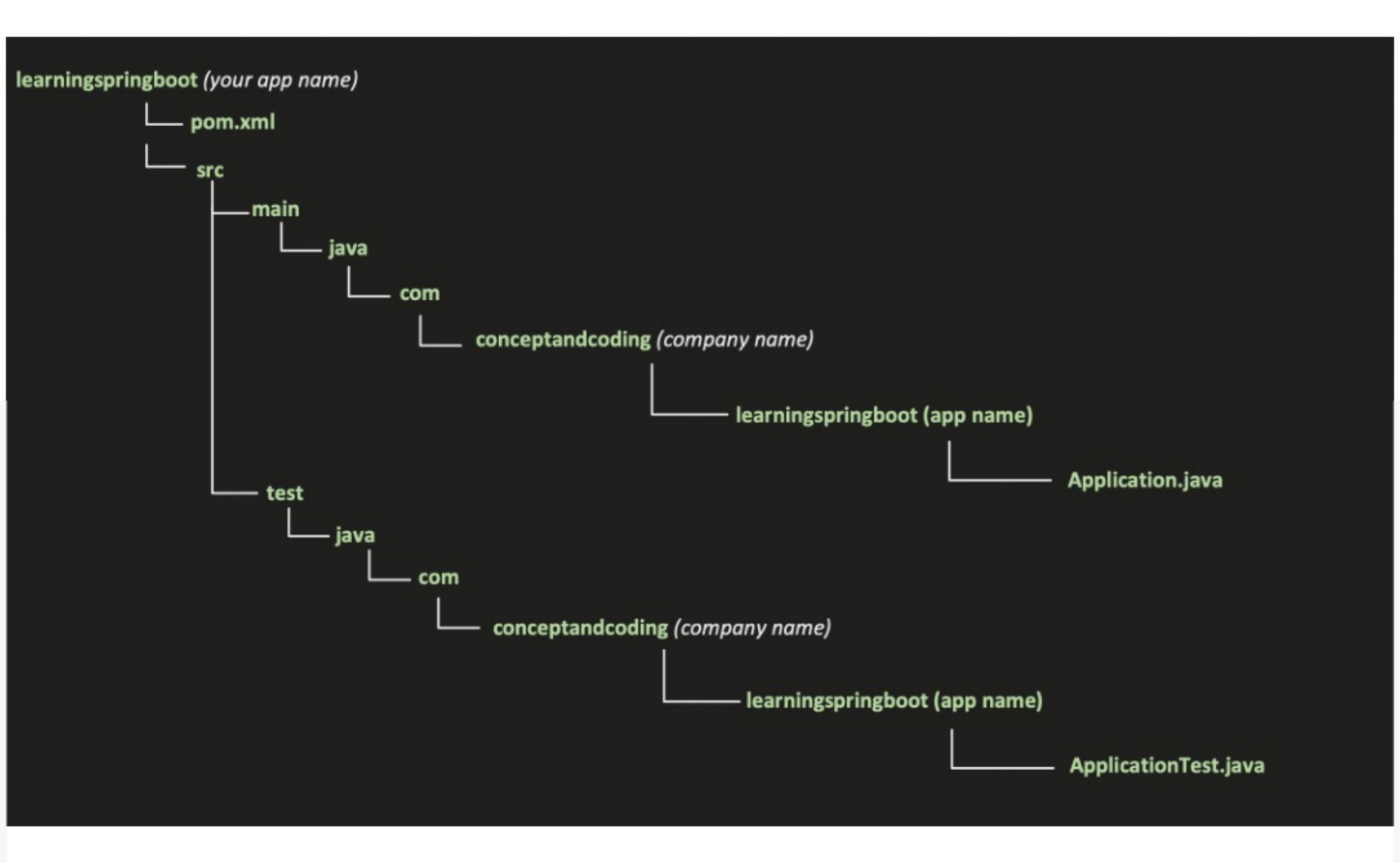
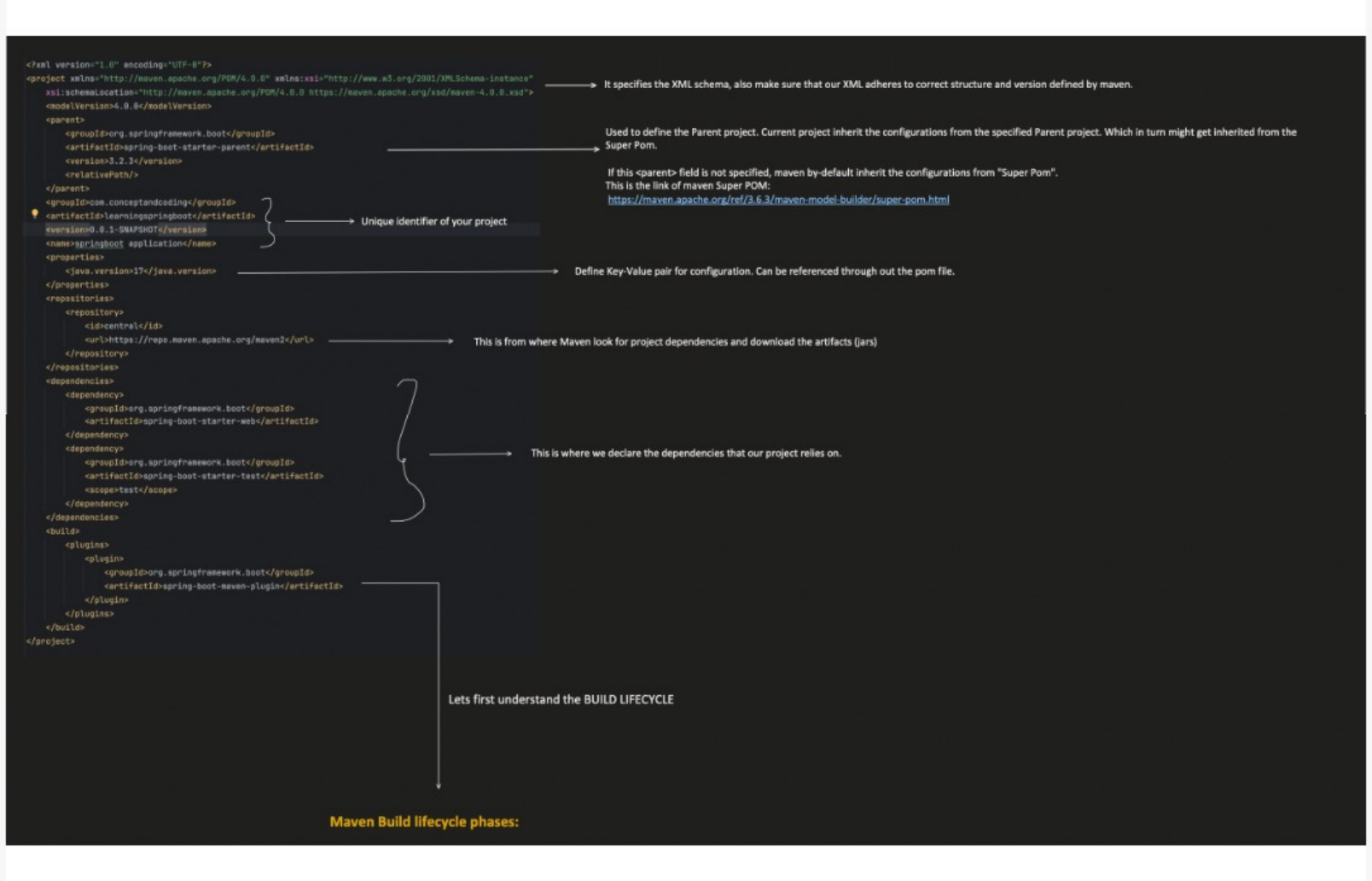
SpringBoot: Maven

"Concept && Coding" YT Video Notes

What is Maven:

- It's a project management tool. Helps developers with:
  - Build generation
  - Dependency resolution
  - Documentation etc.
- Maven uses POM (Project Object Model) to achieve this.
- When "maven" command is given, it looks for "pom.xml" in the current directory & get needed configuration.


learningspringboot (your app name)
pom.xml
src
main
java
com
conceptandcoding (company name)
learningspringboot (app name)
Application.java
test
java
com
conceptandcoding (company name)
learningspringboot (app name)
ApplicationTest.java



```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.3</version>
        <relativePath/>
    </parent>
    <groupId>com.conceptandcoding</groupId>
    <artifactId>learningspringboot</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>springboot application</name>
    <properties>
        <java.version>17</java.version>
    </properties>
    <repositories>
        <repository>
            <id>central</id>
            <url>https://repo.maven.apache.org/maven2</url>
        </repository>
    </repositories>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

It specifies the XML schema, also make sure that our XML adheres to correct structure and version defined by maven.

Used to define the Parent project. Current project inherit the configurations from the specified Parent project. Which in turn might get inherited from the Super Pom.

If this <parent> field is not specified, maven by-default inherit the configurations from "Super Pom".
This is the link of maven Super POM:
https://maven.apache.org/ref/3.6.3/maven-model-builder/super-pom.html

Unique identifier of your project

Define Key-Value pair for configuration. Can be referenced through out the pom file.

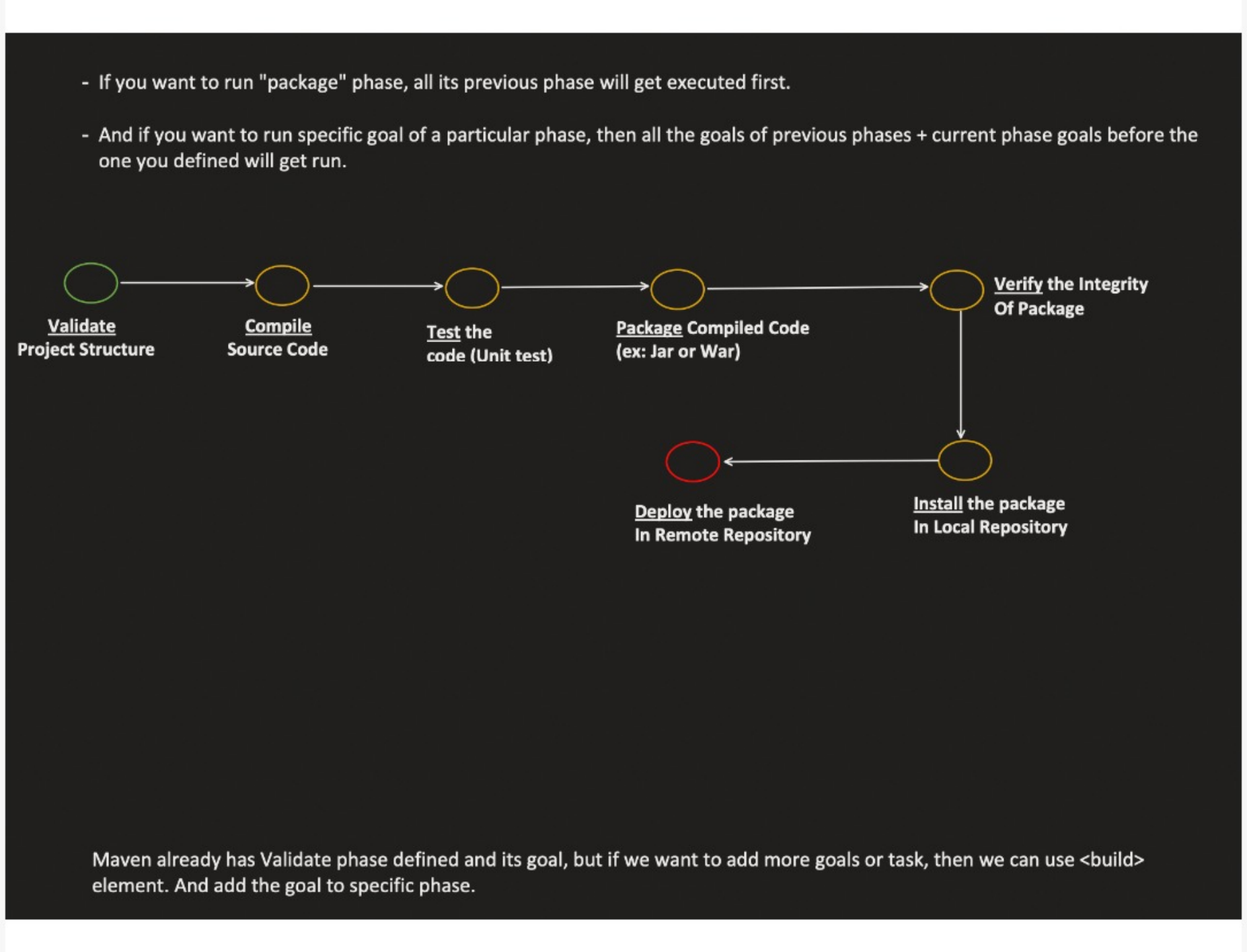
This is from where Maven look for project dependencies and download the artifacts (jars)

This is where we declare the dependencies that our project relies on.

Lets first understand the BUILD LIFECYCLE

Maven Build lifecycle phases:

- If you want to run "package" phase, all its previous phase will get executed first.
- And if you want to run specific goal of a particular phase, then all the goals of previous phases + current phase goals before the one you defined will get run.

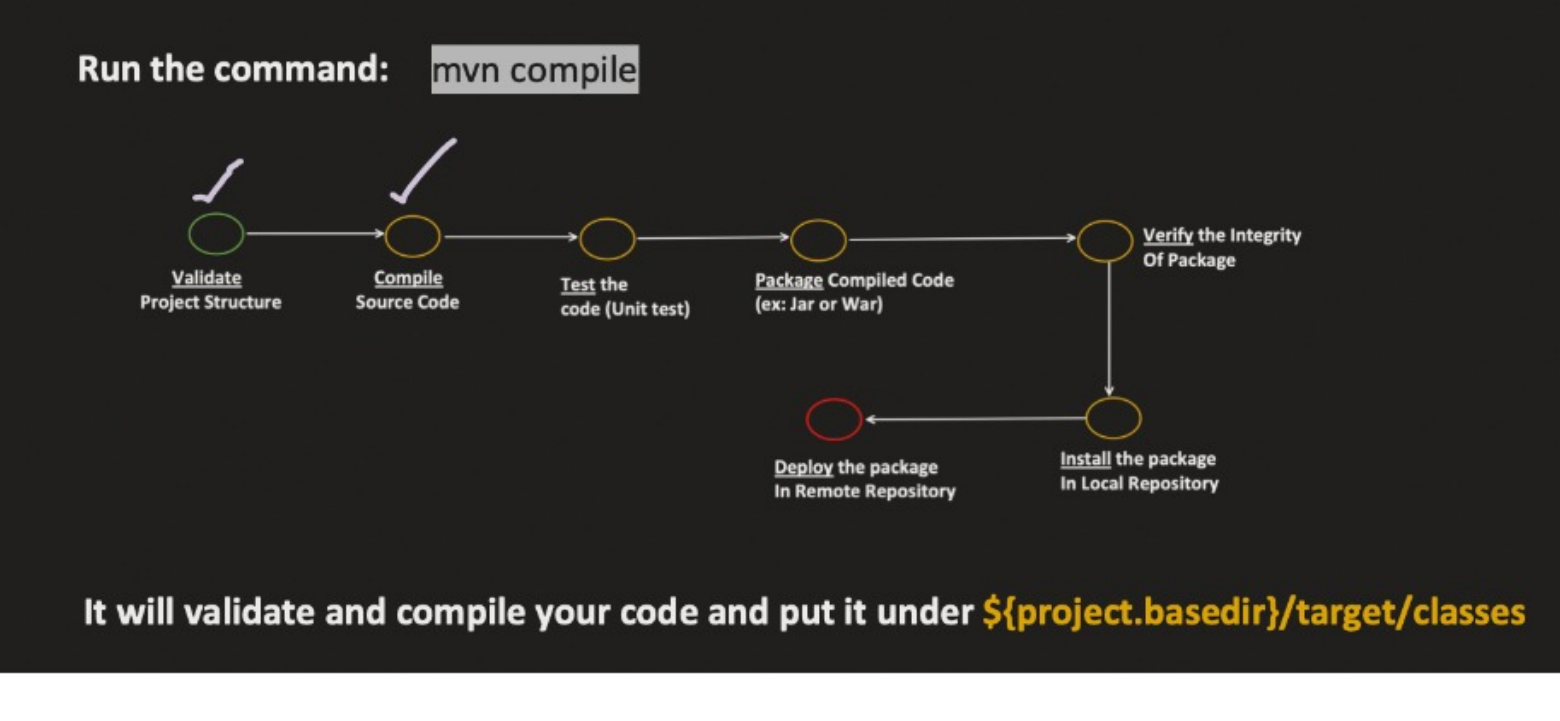

Validate Project Structure
Compile Source Code
Test the code (Unit test)
Package Compiled Code (ex: Jar or War)
Verify the Integrity Of Package
Install the package In Local Repository
Deploy the package In Remote Repository

Maven already has Validate phase defined and its goal, but if we want to add more goals or task, then we can use <build> element. And add the goal to specific phase.

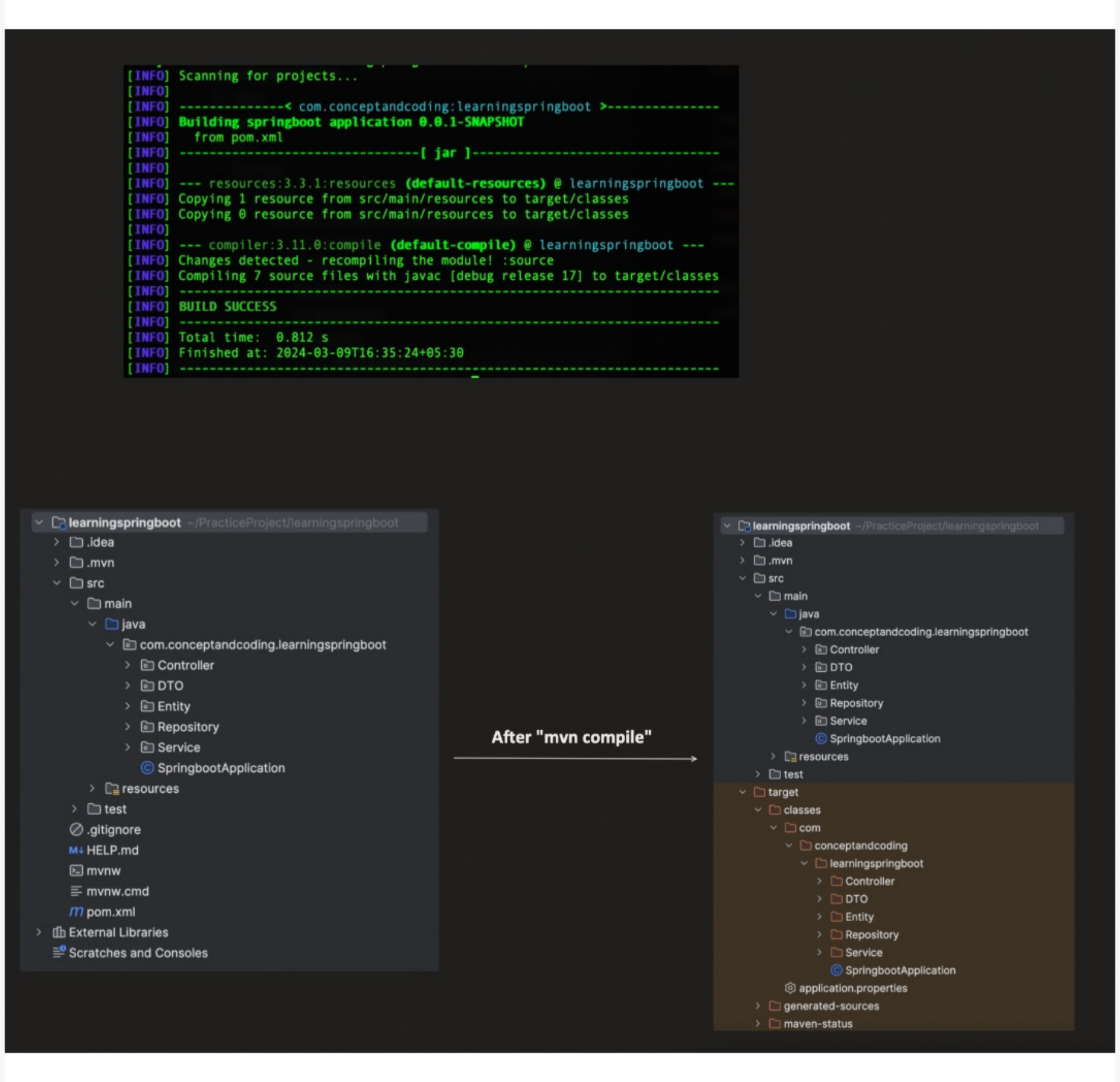
Validate : `mvn validate`

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-checkstyle-plugin</artifactId>
    <version>3.1.2</version>
    <executions>
        <execution>
            <id>validate-checkstyle</id>
            <phase>validate</phase>
            <goals>
                <goal>check</goal>
            </goals>
        </execution>
    </executions>
    <configuration>
        <configLocation>myCodeStyle.xml</configLocation>
    </configuration>
</plugin>
```

Compile: Run the command: `mvn compile`


Validate Project Structure
Compile Source Code
Test the code (Unit test)
Package Compiled Code (ex: Jar or War)
Verify the Integrity Of Package
Install the package In Local Repository
Deploy the package In Remote Repository

It will validate and compile your code and put it under ${project.basedir}/target/classes

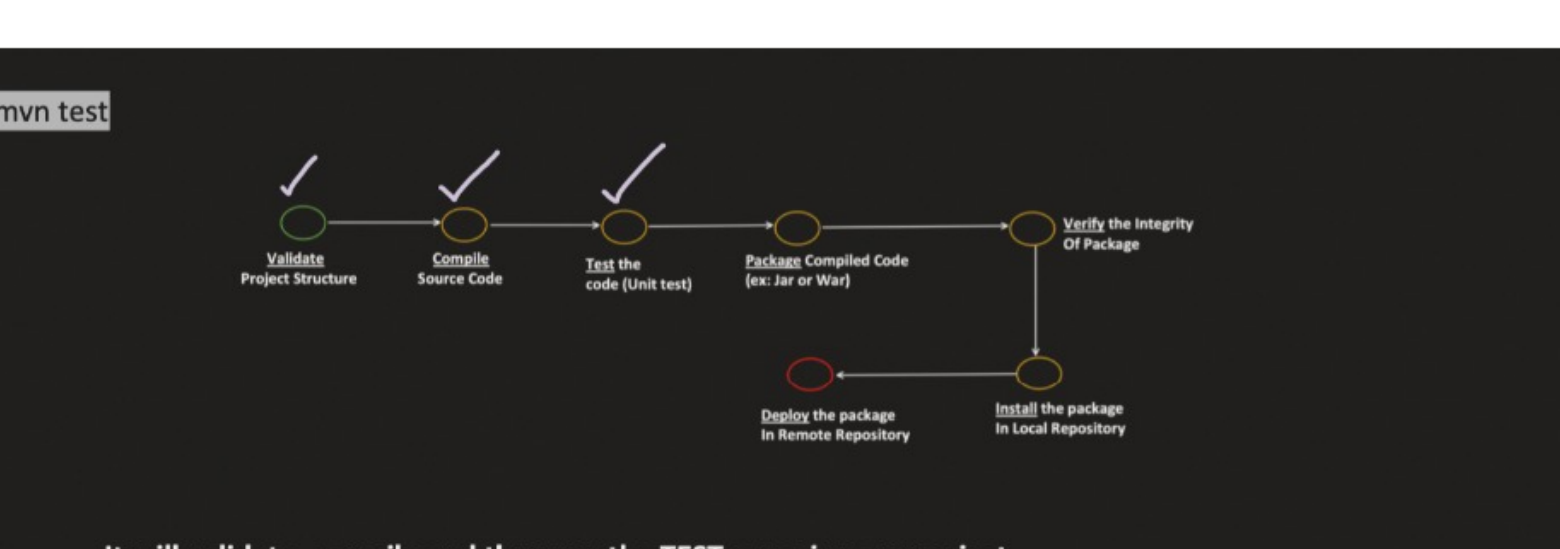


```
[INFO] Scanning for projects...
[INFO]
[INFO] --------------< com.conceptandcoding:learningspringboot >---------------
[INFO] Building springboot application 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ learningspringboot ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ learningspringboot ---
[INFO] Changes detected - recompiling the module! :source
[INFO] Compiling 7 source files with javac [debug release 17] to target/classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.812 s
[INFO] Finished at: 2024-03-09T16:35:24+05:30
[INFO] ------------------------------------------------------------------------
```

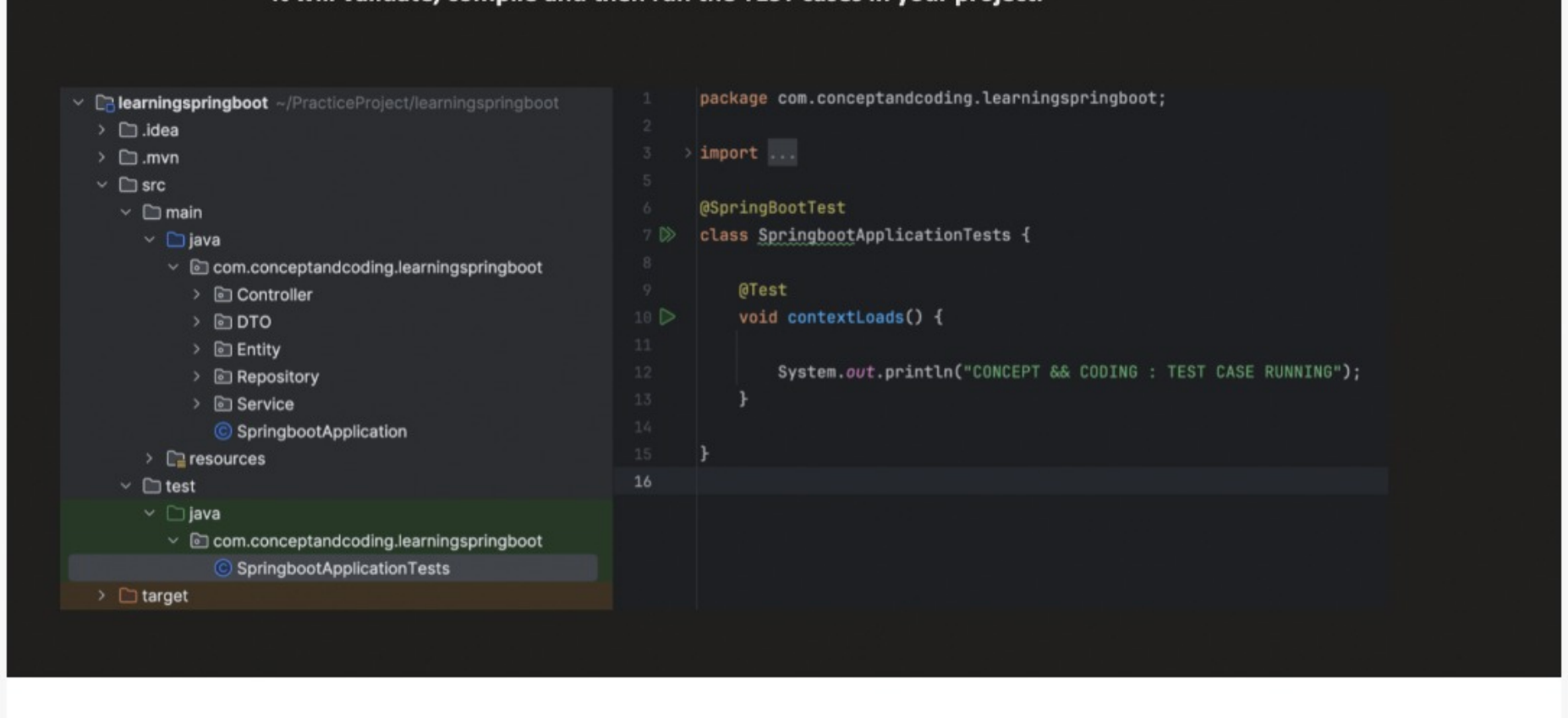
Previously, "Ant" was popular, there we have to tell what to do and also how to do.

```
<project default="compile">
<target name="compile">
//some other properties here
<javac destdir="{provide your destination directory}">
<src>
<pathelement location="src/main/java"/>
</src>
//some other properties here
</javac>
</target>
</project>
```

Test: `mvn test`

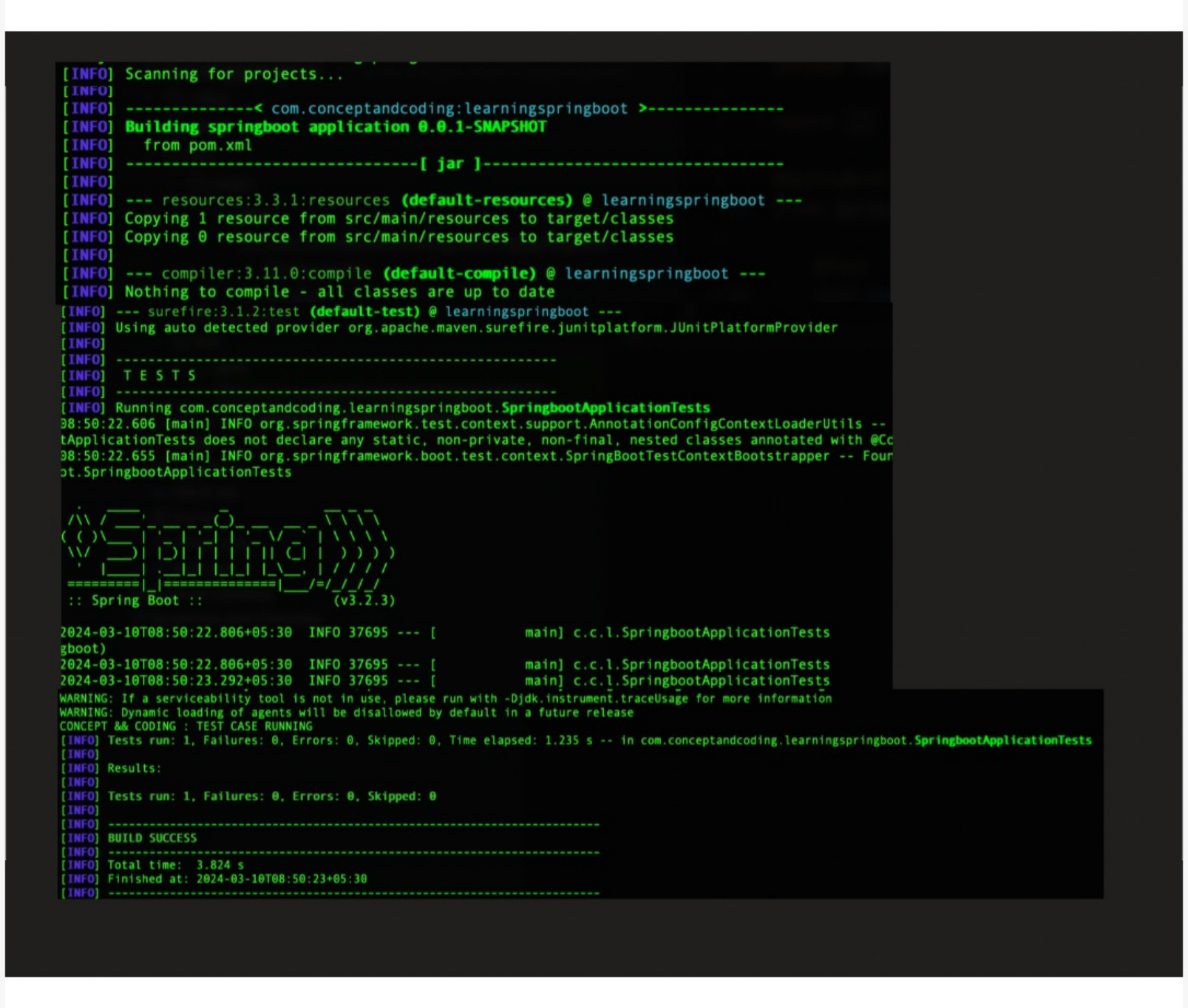

Validate Project Structure
Compile Source Code
Test the code (Unit test)
Package Compiled Code (ex: Jar or War)
Verify the Integrity Of Package
Install the package In Local Repository
Deploy the package In Remote Repository

It will validate, compile and then run the TEST cases in your project.



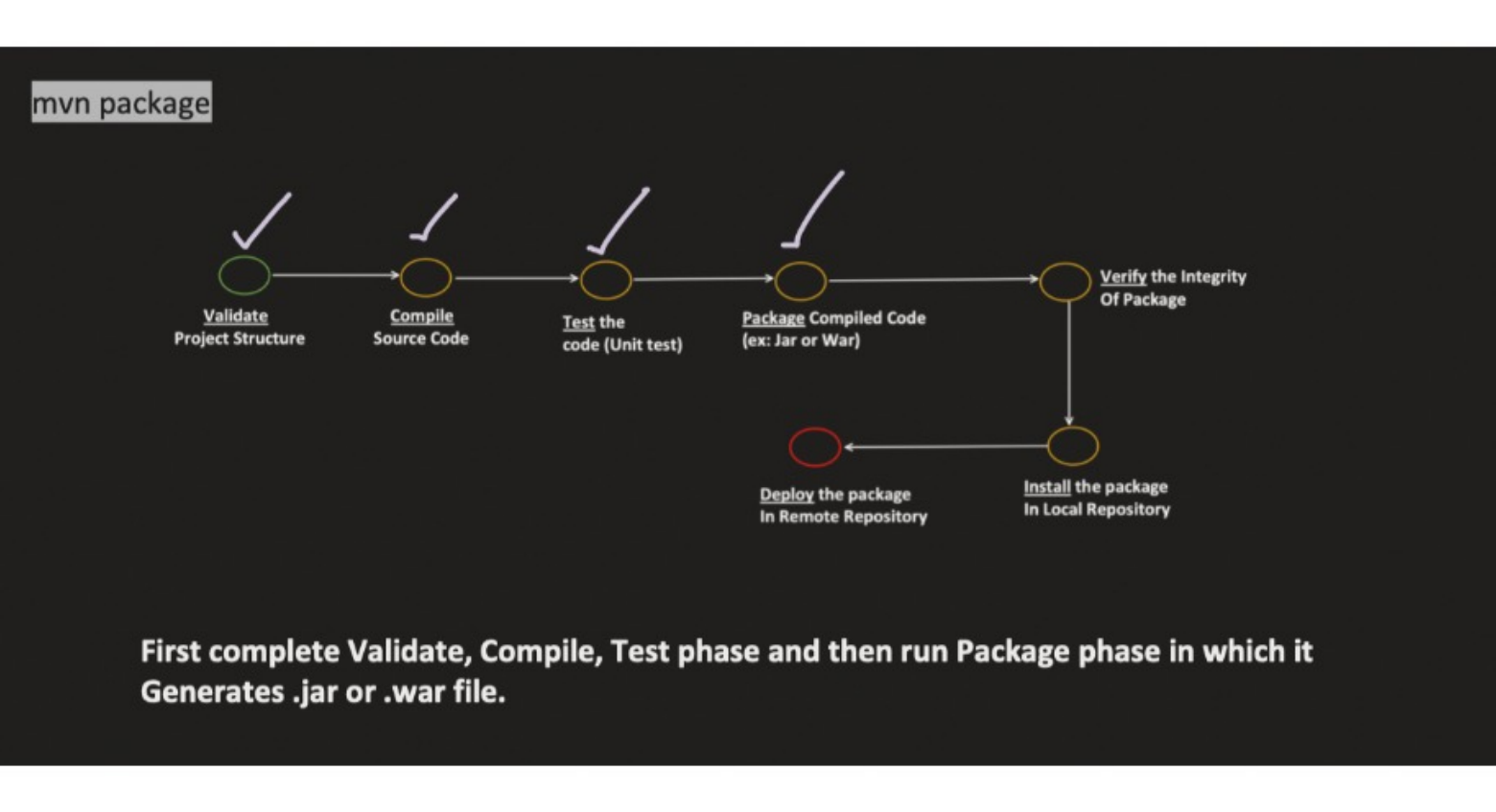
```
package com.conceptandcoding.learningspringboot;

import ...

@SpringBootTest
class SpringbootApplicationTests {

    @Test
    void contextLoads() {

        System.out.println("CONCEPT && CODING : TEST CASE RUNNING");
    }

}
```

learningspringboot ~/PracticeProject/learningspringboot
.idea
.mvn
src
main
java
com.conceptandcoding.learningspringboot
Controller
DTO
Entity
Repository
Service
SpringbootApplication
resources
test
java
com.conceptandcoding.learningspringboot
SpringbootApplicationTests
target

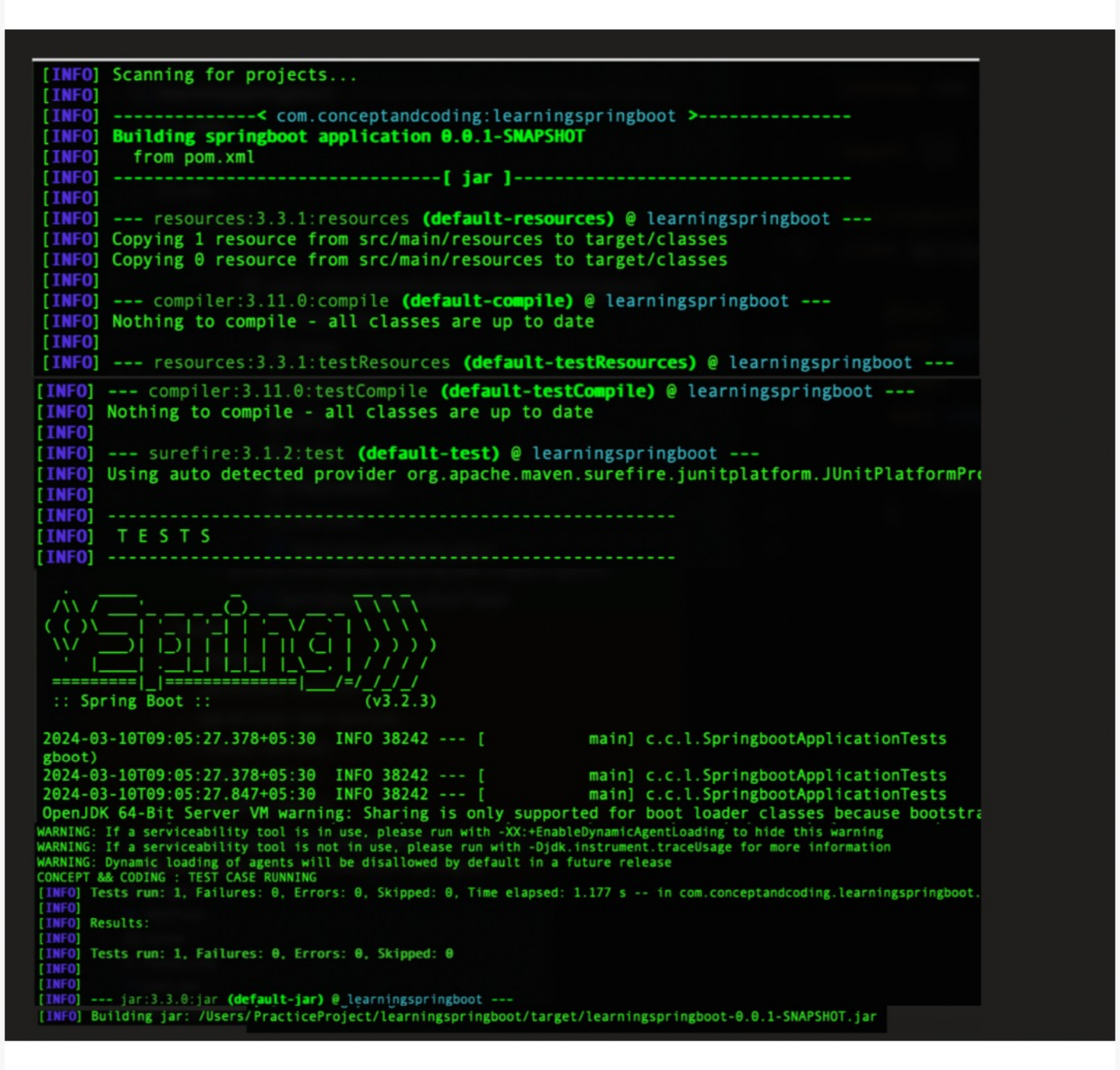


```
[INFO] Scanning for projects...
[INFO]
[INFO] --------------< com.conceptandcoding:learningspringboot >---------------
[INFO] Building springboot application 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ learningspringboot ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ learningspringboot ---
[INFO] Nothing to compile - all classes are up to date
[INFO] --- surefire:3.1.2:test (default-test) @ learningspringboot ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.conceptandcoding.learningspringboot.SpringbootApplicationTests
08:50:22.606 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils --
tApplicationTests does not declare any static, non-private, non-final, nested classes annotated with @Co
08:50:22.655 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper -- Foun
ot.SpringbootApplicationTests

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.2.3)

2024-03-10T08:50:22.806+05:30  INFO 37695 --- [           main] c.c.l.SpringbootApplicationTests
gboot)
2024-03-10T08:50:22.806+05:30  INFO 37695 --- [           main] c.c.l.SpringbootApplicationTests
2024-03-10T08:50:23.292+05:30  INFO 37695 --- [           main] c.c.l.SpringbootApplicationTests
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
CONCEPT && CODING : TEST CASE RUNNING
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.235 s -- in com.conceptandcoding.learningspringboot.SpringbootApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.824 s
[INFO] Finished at: 2024-03-10T08:50:23+05:30
[INFO] ------------------------------------------------------------------------
```

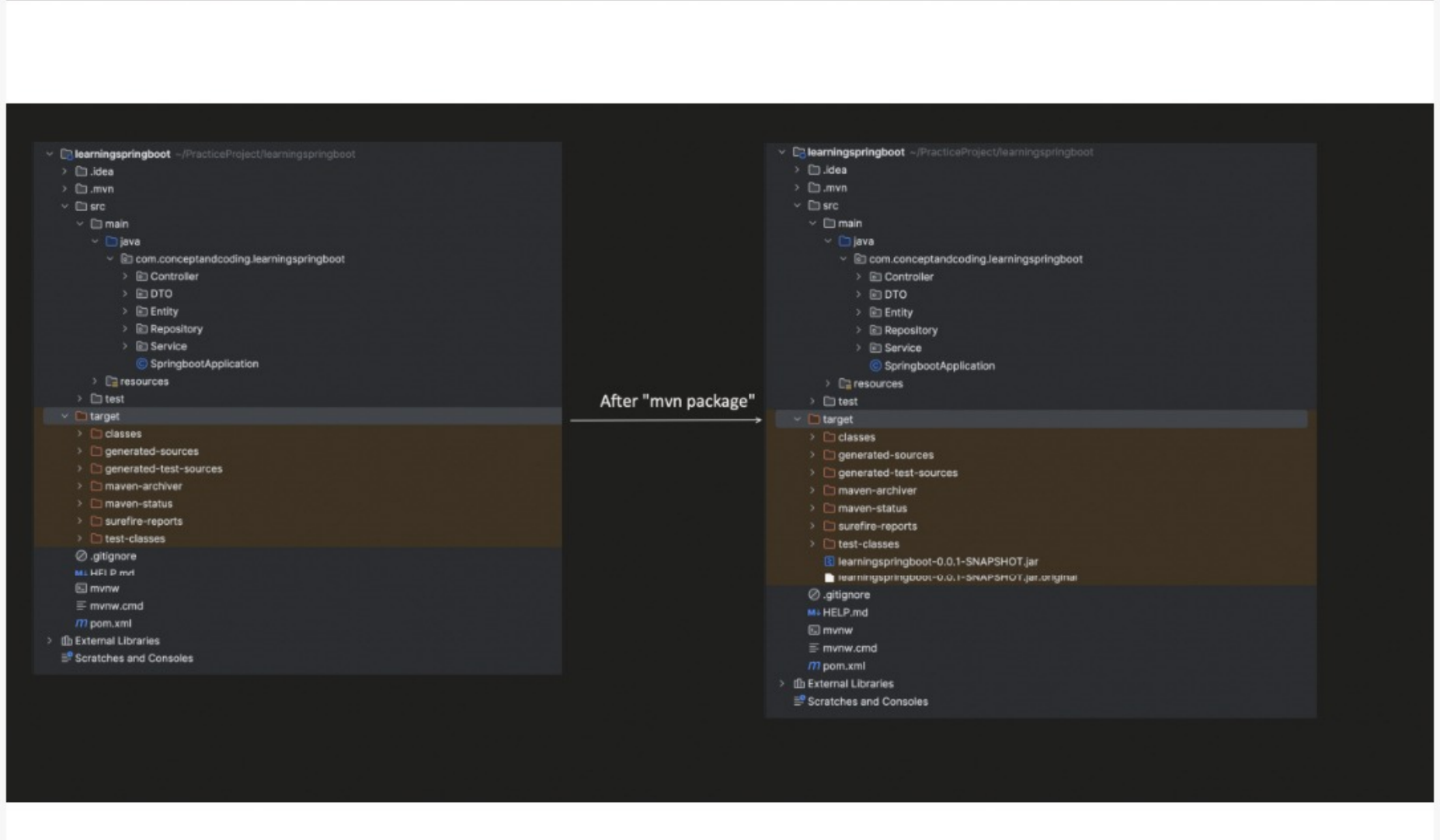
Package: `mvn package`


Validate Project Structure
Compile Source Code
Test the code (Unit test)
Package Compiled Code (ex: Jar or War)
Verify the Integrity Of Package
Install the package In Local Repository
Deploy the package In Remote Repository

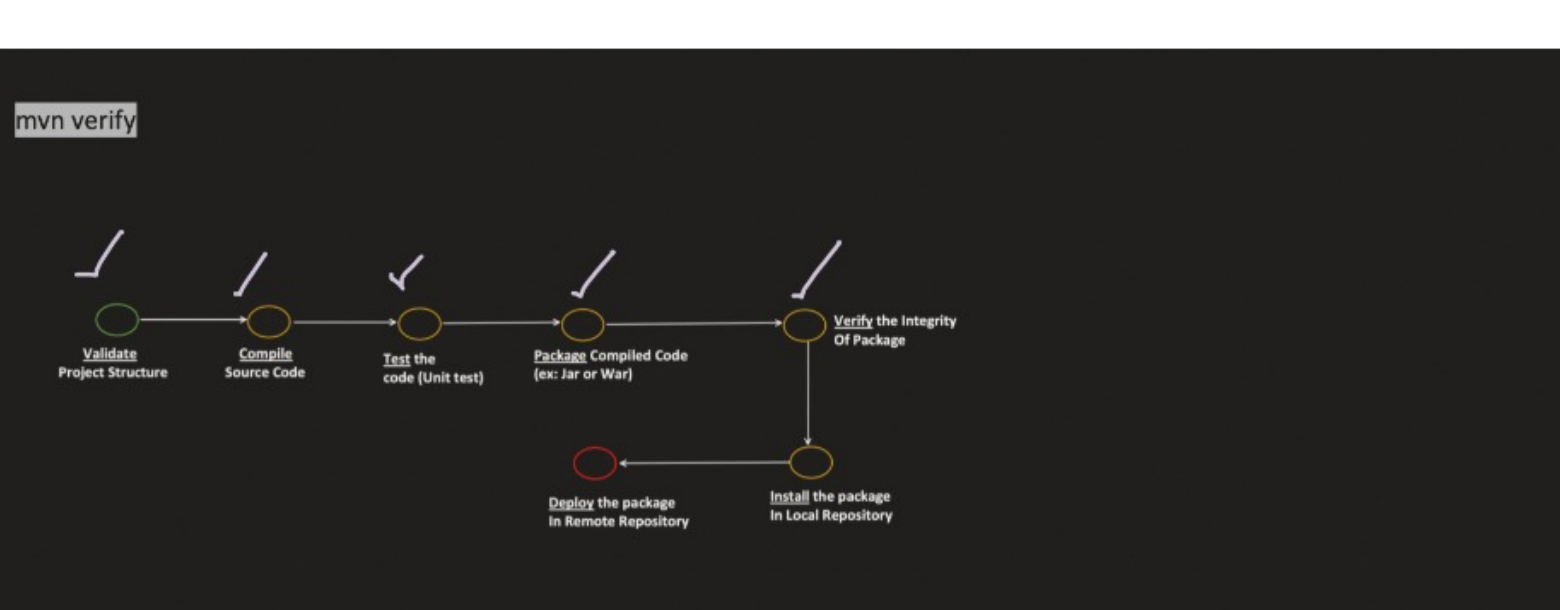
First complete Validate, Compile, Test phase and then run Package phase in which it Generates .jar or .war file.



```
[INFO] Scanning for projects...
[INFO]
[INFO] --------------< com.conceptandcoding:learningspringboot >---------------
[INFO] Building springboot application 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ learningspringboot ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ learningspringboot ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ learningspringboot ---
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ learningspringboot ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- surefire:3.1.2:test (default-test) @ learningspringboot ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformPr
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.2.3)

2024-03-10T09:05:27.378+05:30  INFO 38242 --- [           main] c.c.l.SpringbootApplicationTests
gboot)
2024-03-10T09:05:27.378+05:30  INFO 38242 --- [           main] c.c.l.SpringbootApplicationTests
2024-03-10T09:05:27.847+05:30  INFO 38242 --- [           main] c.c.l.SpringbootApplicationTests
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstra
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
CONCEPT && CODING : TEST CASE RUNNING
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.177 s -- in com.conceptandcoding.learningspringboot.
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ learningspringboot ---
[INFO] Building jar: /Users/PracticeProject/learningspringboot/target/learningspringboot-0.0.1-SNAPSHOT.jar
```



Verify: `mvn verify`


Validate Project Structure
Compile Source Code
Test the code (Unit test)
Package Compiled Code (ex: Jar or War)
Verify the Integrity Of Package
Install the package In Local Repository
Deploy the package In Remote Repository

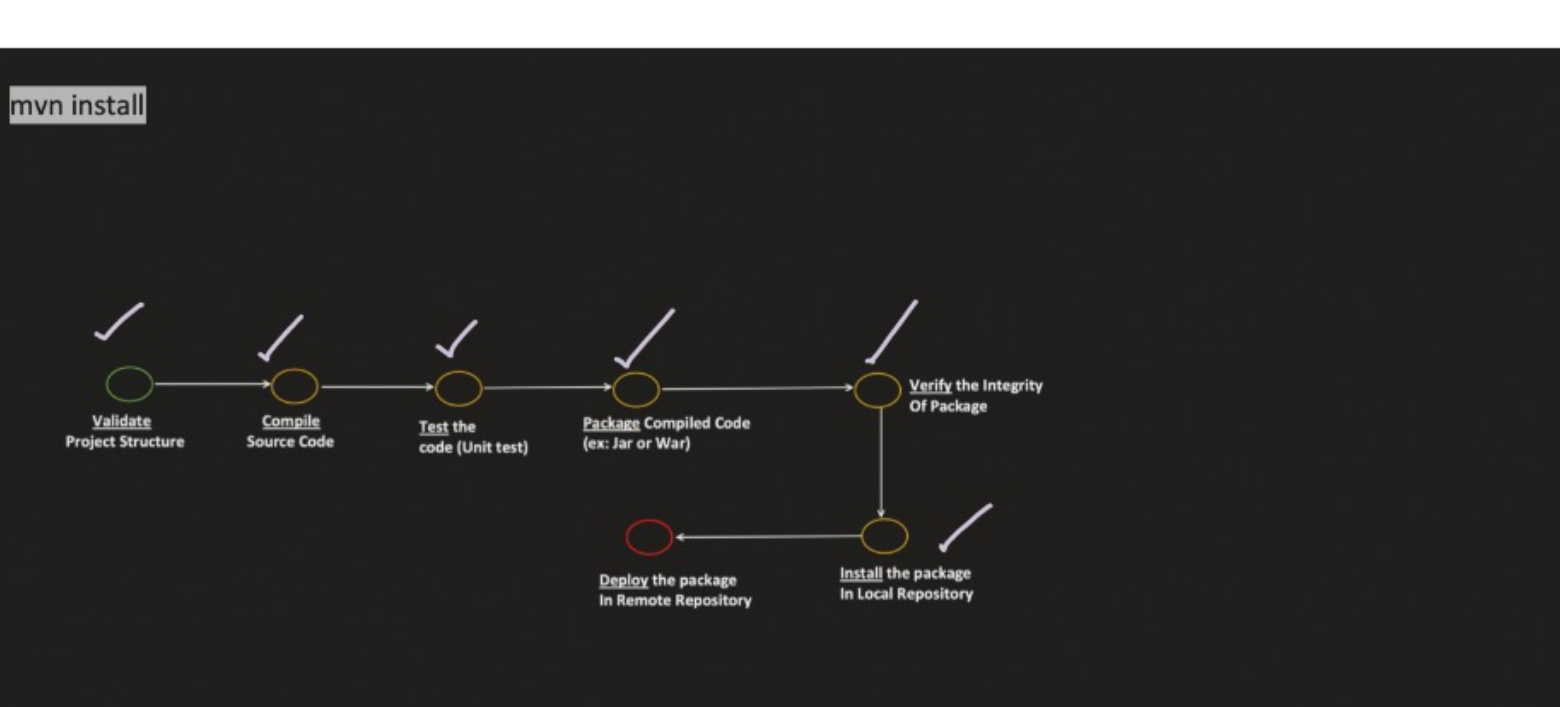
It can perform some additional checks apart from unit test cases like:
- STATIC CODE ANALYSIS
- CHECKSUM VERFICATION etc...

STATIC CODE ANALYSIS:

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-pmd-plugin</artifactId>
    <version>3.21.2</version>
    <executions>
        <execution>
            <id>pmd-analysis</id>
            <phase>verify</phase>
            <goals>
                <goal>pmd</goal>
            </goals>
        </execution>
    </executions>
</plugin>
```

PMD is a source code analyzer:
- Finds unused variable
- Finds unused imports
- Empty Catch block
- No usage of object
- Finds duplicate code etc...

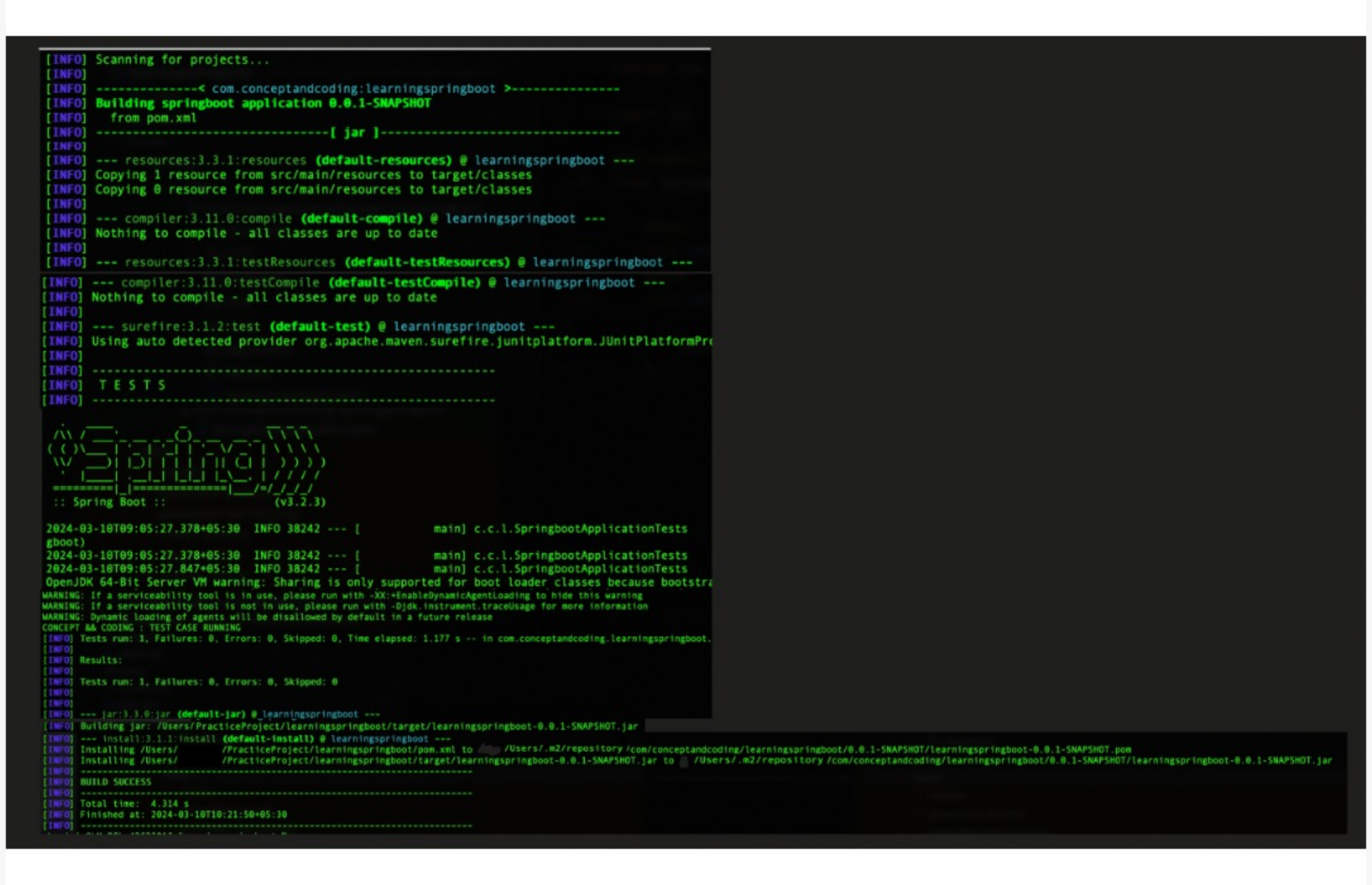
Install: `mvn install`


Validate Project Structure
Compile Source Code
Test the code (Unit test)
Package Compiled Code (ex: Jar or War)
Verify the Integrity Of Package
Install the package In Local Repository
Deploy the package In Remote Repository

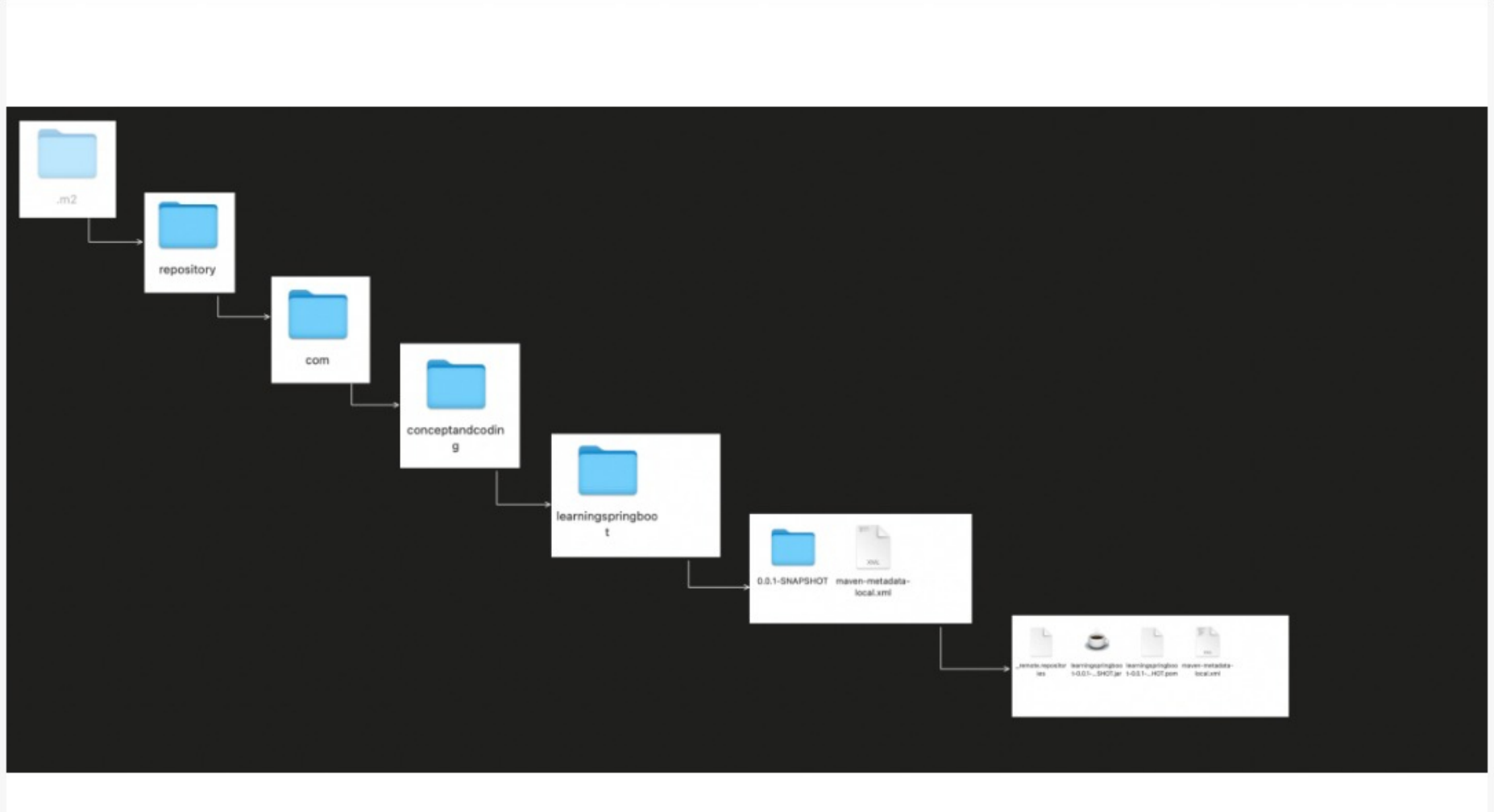
It will install the .jar package in local Maven Repository.
which is typically located in your user home directory (~/.m2/repository)

settings.xml (in .m2 folder)

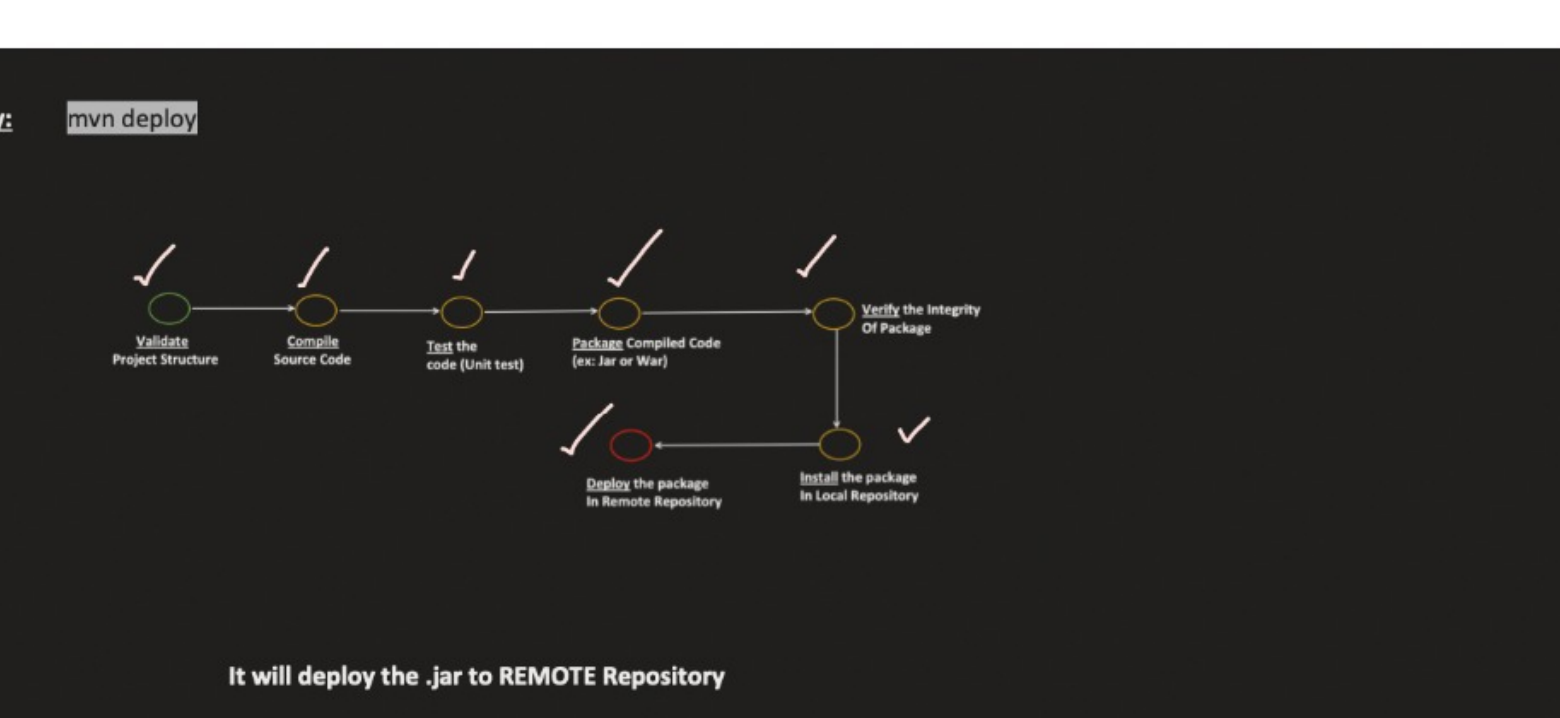
```
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.0
                              http://maven.apache.org/xsd/settings-1.0.0.xsd">

   <!-- Local Repository: Location where Maven stores downloaded artifacts -->
   <localRepository>${user.home}/.m2/repository</localRepository>


  <!— some other configurations goes here —>

</settings>
```



```
[INFO] Scanning for projects...
[INFO]
[INFO] --------------< com.conceptandcoding:learningspringboot >---------------
[INFO] Building springboot application 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ learningspringboot ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ learningspringboot ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ learningspringboot ---
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ learningspringboot ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- surefire:3.1.2:test (default-test) @ learningspringboot ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformPr
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------

 :: Spring Boot ::                (v3.2.3)

2024-03-10T09:05:27.378+05:30  INFO 38242 --- [           main] c.c.l.SpringbootApplicationTests
gboot)
2024-03-10T09:05:27.378+05:30  INFO 38242 --- [           main] c.c.l.SpringbootApplicationTests
2024-03-10T09:05:27.847+05:30  INFO 38242 --- [           main] c.c.l.SpringbootApplicationTests
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstra
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
CONCEPT && CODING : TEST CASE RUNNING
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.177 s -- in com.conceptandcoding.learningspringboot.
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ learningspringboot ---
[INFO] Building jar: /Users/PracticeProject/learningspringboot/target/learningspringboot-0.0.1-SNAPSHOT.jar
[INFO] --- install:3.1.1:install (default-install) @ learningspringboot ---
[INFO] Installing /Users/       /PracticeProject/learningspringboot/pom.xml to    /Users/.m2/repository/com/conceptandcoding/learningspringboot/0.0.1-SNAPSHOT/learningspringboot-0.0.1-SNAPSHOT.pom
[INFO] Installing /Users/       /PracticeProject/learningspringboot/target/learningspringboot-0.0.1-SNAPSHOT.jar to    /Users/.m2/repository/com/conceptandcoding/learningspringboot/0.0.1-SNAPSHOT/learningspringboot-0.0.1-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.314 s
[INFO] Finished at: 2024-03-10T10:21:50+05:30
[INFO] ------------------------------------------------------------------------
```



Deploy: `mvn deploy`


Validate Project Structure
Compile Source Code
Test the code (Unit test)
Package Compiled Code (ex: Jar or War)
Verify the Integrity Of Package
Install the package In Local Repository
Deploy the package In Remote Repository

It will deploy the .jar to REMOTE Repository

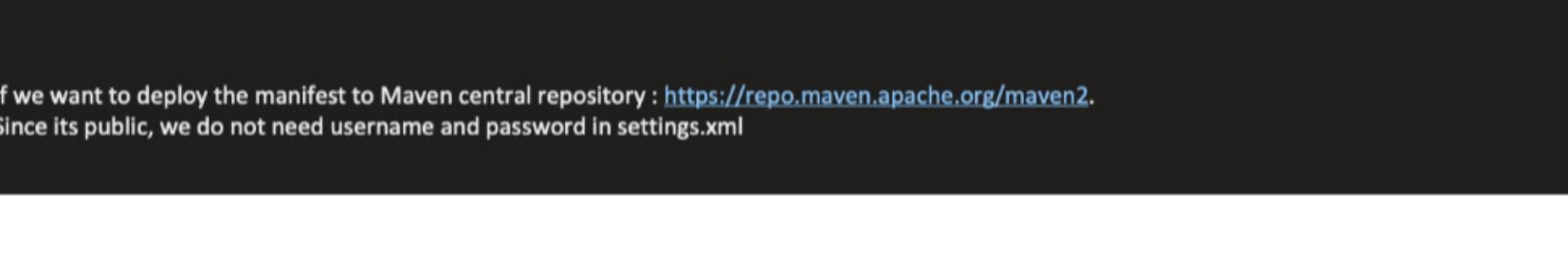
pom.xml

```
<project>
    <!-- ... other project configurations ... -->

    <distributionManagement>
        <repository>
            <id>remote repository id</id>
            <url>https://remote-repository-url</url>
        </repository>
    </distributionManagement>

    <!-- ... other pom.xml configurations ... -->
</project>
```

settings.xml

```
<settings>
    <!--  other settings configurations  -->

    <servers>
        <server>
            <id>remote repository id</id>
            <username>remote-repository-username</username>
            <password>remote-repository-password</password>
        </server>
    </servers>

    <!--  other settings configurations -->
</settings>
```

If we do not define the remote repository, then MAVEN during "mvn deploy" command we will face below error



```
[INFO] --- deploy:3.1.1:deploy (default-deploy) @ learningspringboot ---
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.587 s
[INFO] Finished at: 2024-03-10T13:00:11+05:30
[INFO] ------------------------------------------------------------------------
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-deploy-plugin:3.1.1:deploy (default-deploy) on project learningspringboot: Deployment failed: repository element was not specified in the POM inside distributionManagement element or
in -DaltDeploymentRepository=id::url parameter -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoExecutionException
```

If we want to deploy the manifest to Maven central repository : https://repo.maven.apache.org/maven2.
Since its public, we do not need username and password in settings.xml